# Assignment-5

* Problem Statement

Department of computer engineering has student's club named 'Pinnacle club'. Student of second, third and final year of department can be granted membership on request. Similarly, one may cancel the membership of club. First node is reserved for president of club and last node is reserved for secretary of club. Write C program to maintain club member's information using singly linked list. Store student PRN and name. Write function to

a) Add & delete the members as well as president or even secretary.
b) Compute total number of members of club.
c) Display members
d) Sorting of two linked list.
e) Merging of two linked list.
f) Reversing using the pointers

* Objectives

(1) To understand the concept of singly linked list and dynamic memory allocation using pointers in C.

(2) To perform Insertion, deletion, display, count, nodes, sort, reverse, operations on singly linked list.

③ To develop a menu driven application for workshop enrollment management.

* Pseudo Code
write pseudo code for insertion, deletion, display, count nodes, sort, reverse using SLL.

1. Insertion
(i) Create a new node and enter PRN and name.
(ii) If the list is empty set, head = newnode
(iii) Otherwise, traverse to the last node and link it to newnode.
(iv) Set newnode -> next = NULL.

2. Deletion.
(i) Enter PRN to delete
(ii) Search for the node containing the PRN.
(iii) If it is the first node, update head
(iv) Otherwise, link the previous node to the next node and free the deleted node.

3. Display
(i) Set temp = head
(ii) Traverse until temp == Null.
(iii) Display PRN and name of each node

4. Count nodes
(i) Set count = 0 and temp = head.

(ii) Traverse the list, incrementing count for each node.
(iii) Display count.

5. Sorting
(i) Compare PRN's of adjacent nodes.
(ii) Swap their data if they in the wrong order.
(iii) Repeat until the list is sorted.

6. Reversing
(i) Set prev = Null, current = head
(ii) Repeat while current != Null:
next = current -> next
current->next = prev
prev = current
current = next.
(iii) Set head = prev.

* Time complexity

| Operation | Time Complexity |
|---|---|
| Insertion in beginning | O(1) |
| Insertion at end | O(n) |
| ~~Insertion~~ deletion (begin) | O(1) |
| Deletion at end | O(n) |
| Display | O(n) |
| Count nodes | O(n) |
| Sorting | $O(n^2)$ |
| Reversing | O(n) |
| Merging list | O(n+m) |

* Conclusion

The program demonstrate dynamic memory allocation & efficient manipulation of linked list data structures - Operations such as insertion, deletion, traversal, counting, sorting and reversing were successfully implemented.

* FAQ's

1. Why is a linked list preferred over an array in this application

→ (i) Dynamic Size: Members can be added or removed easily.

(ii) Easy Insertion & Deletion: No need to shift elements.

(iii) Memory efficient: Memory is allocated as needed.

(iv) Flexible Management: President & Secretary can be added or deleted.

2. What information is stored in each node of circular linked list?

→ (i) Data: Student PRN and Name.

(ii) Pointer: Address of the next node, which points back to first node in the last node.

3. What is singly linked list?

→ It is linear data structure in which each node contains

data & pointer to next node. The last node points to null.

4. What information is stored in each node?
→ Each node contains student PRN, name, & pointer to the next node.

5. What is dynamic memory allocation?
→ Dynamic memory is process of allocating memory during program execution as needed, using functions like malloc() & calloc().

6. Which function is used to dealloc. release allocated memory?
→ The free() function is used to release dynamically allocated memory in C.

7. What is the time complexity of inserting a node at the beginning?
→ For inserting a node at beginning the time complexity is O(1) (constant time).

8. What is time complexity of searching for student?
→ Time complexity of searching for a student in linked list is O(n) (linear time).

9. How is linked list reversed?
→ A linked list is reversed by changing the direction of pointers using three pointers : prev, current & next.

10. Which sorting technique can be used in linked list.?
→ Bubble sort can be used to sort the nodes of a linked list.

# PRACTICAL 5

# DATA STRUCTURE

Pinnacle Club Management System

Singly Linked List in C

Program Code and GDB Input/Output Screenshot

# Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Node {
    long long prn;
    char name[50];
    struct Node *next;
};
struct Node *head = NULL;
/* Create a new node */
struct Node* createNode(long long prn, char name[]) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->prn = prn;
    strcpy(newNode->name, name);
    newNode->next = NULL;
    return newNode;
}
/* Count members */
int countMembers(struct Node *temp) {
    int count = 0;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}
/* Display members */
void display(struct Node *temp) {
    if (temp == NULL) {
        printf("Club is empty!\n");
        return;
    }
    printf("\n--- Pinnacle Club Members ---\n");
    printf("PRN\t\tName\t\tPosition\n");
    int count = countMembers(temp);
    int i = 1;
    while (temp != NULL) {
        printf("%lld\t\t%s\t\t", temp->prn, temp->name);
        if (i == 1)
            printf("President");
        else if (i == count)
            printf("Secretary");
        else
            printf("Member");
        printf("\n");
        temp = temp->next;
        i++;
    }
}
/* Add president at beginning */
void addPresident() {
    long long prn;
    char name[50];
    printf("Enter President PRN: ");
    scanf("%lld", &prn);
    printf("Enter President Name: ");
    scanf(" %49[^\n]", name);
    struct Node *newNode = createNode(prn, name);
    newNode->next = head;
    head = newNode;
    printf("President added successfully!\n");
}
/* Add secretary at end */
void addSecretary() {
    long long prn;
    char name[50];
    printf("Enter Secretary PRN: ");
    scanf("%lld", &prn);
    printf("Enter Secretary Name: ");
    scanf(" %49[^\n]", name);
    struct Node *newNode = createNode(prn, name);
    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
    printf("Secretary added successfully!\n");
```

```
}
/* Add ordinary member before secretary */
void addMember() {
    long long prn;
    char name[50];
    printf("Enter Member PRN: ");
    scanf("%lld", &prn);
    printf("Enter Member Name: ");
    scanf(" %49[^\n]", name);
    struct Node *newNode = createNode(prn, name);
    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL && temp->next->next != NULL)
            temp = temp->next;
        if (temp->next == NULL) {
            temp->next = newNode;
        } else {
            newNode->next = temp->next;
            temp->next = newNode;
        }
    }
    printf("Member added successfully!\n");
}
/* Delete president */
void deletePresident() {
    if (head == NULL) {
        printf("Club is empty!\n");
        return;
    }
    struct Node *temp = head;
    head = head->next;
    printf("Deleted President: %s\n", temp->name);
    free(temp);
}
/* Delete secretary */
void deleteSecretary() {
    if (head == NULL) {
        printf("Club is empty!\n");
        return;
    }
    if (head->next == NULL) {
        printf("Deleted: %s\n", head->name);
        free(head);
        head = NULL;
        return;
    }
    struct Node *temp = head;
    while (temp->next->next != NULL)
        temp = temp->next;
    struct Node *last = temp->next;
    printf("Deleted Secretary: %s\n", last->name);
    temp->next = NULL;
    free(last);
}
/* Delete member by PRN */
void deleteMember() {
    long long prn;
    if (head == NULL) {
        printf("Club is empty!\n");
        return;
    }
    printf("Enter PRN to delete: ");
    scanf("%lld", &prn);
    struct Node *temp = head;
    struct Node *prev = NULL;
    while (temp != NULL && temp->prn != prn) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("PRN not found!\n");
        return;
    }
    if (prev == NULL)
        head = temp->next;
    else
        prev->next = temp->next;
    printf("Deleted member: %s\n", temp->name);
    free(temp);
}
/* Reverse linked list using pointers */
void reverseList() {
    struct Node *prev = NULL;
    struct Node *current = head;
    struct Node *nextNode = NULL;
```

```
    while (current != NULL) {
        nextNode = current->next;
        current->next = prev;
        prev = current;
        current = nextNode;
    }
    head = prev;
    printf("Linked list reversed!\n");
}
/* Sort list by PRN using bubble sort of data */
void sortList(struct Node *start) {
    if (start == NULL)
        return;
    struct Node *i, *j;
    long long tempPRN;
    char tempName[50];
    for (i = start; i->next != NULL; i = i->next) {
        for (j = start; j->next != NULL; j = j->next) {
            if (j->prn > j->next->prn) {
                tempPRN = j->prn;
                j->prn = j->next->prn;
                j->next->prn = tempPRN;
                strcpy(tempName, j->name);
                strcpy(j->name, j->next->name);
                strcpy(j->next->name, tempName);
            }
        }
    }
}
/* Merge two lists by connecting pointers */
struct Node* mergeLists(struct Node *first, struct Node *second) {
    if (first == NULL)
        return second;
    struct Node *temp = first;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = second;
    return first;
}
/* Read a separate linked list */
struct Node* createList() {
    struct Node *newHead = NULL;
    struct Node *tail = NULL;
    int n, i;
    long long prn;
    char name[50];
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter PRN: ");
        scanf("%lld", &prn);
        printf("Enter Name: ");
        scanf(" %49[^\n]", name);
        struct Node *newNode = createNode(prn, name);
        if (newHead == NULL)
            newHead = newNode;
        else
            tail->next = newNode;
        tail = newNode;
    }
    return newHead;
}
/* Free all nodes */
void freeList(struct Node *temp) {
    while (temp != NULL) {
        struct Node *nextNode = temp->next;
        free(temp);
        temp = nextNode;
    }
}
int main() {
    int choice;
    struct Node *list2 = NULL;
    do {
        printf("\n===== PINNACLE CLUB MENU =====\n");
        printf("1. Add President\n");
        printf("2. Add Member\n");
        printf("3. Add Secretary\n");
        printf("4. Delete President\n");
        printf("5. Delete Member by PRN\n");
        printf("6. Delete Secretary\n");
        printf("7. Count Members\n");
        printf("8. Display Members\n");
        printf("9. Sort Members by PRN\n");
        printf("10. Create Second List\n");
        printf("11. Merge Two Lists\n");
        printf("12. Reverse List\n");
```

```
        printf("13. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                addPresident();
                break;
            case 2:
                addMember();
                break;
            case 3:
                addSecretary();
                break;
            case 4:
                deletePresident();
                break;
            case 5:
                deleteMember();
                break;
            case 6:
                deleteSecretary();
                break;
            case 7:
                printf("Total members: %d\n",
                       countMembers(head));
                break;
            case 8:
                display(head);
                break;
            case 9:
                sortList(head);
                printf("List sorted by PRN!\n");
                break;
            case 10:
                if (list2 != NULL) {
                    printf("Second list already exists. "
                           "Merge it before creating another.\n");
                } else {
                    printf("Create Second Linked List:\n");
                    list2 = createList();
                }
                break;
            case 11:
                if (list2 == NULL) {
                    printf("Create the second list first!\n");
                } else {
                    head = mergeLists(head, list2);
                    list2 = NULL;
                    printf("Lists merged successfully!\n");
                }
                break;
            case 12:
                reverseList();
                break;
            case 13:
                freeList(head);
                freeList(list2);
                head = NULL;
                list2 = NULL;
                printf("Exiting program. Goodbye!\n");
                break;
            default:
                printf("Invalid choice! Try again.\n");
        }
    } while (choice != 13);
    return 0;
}
```

# GDB / Program Input and Output Screenshot

